

Trabajo Final Integrador - Laboratorio de Python

Algoritmos y Estructuras de Datos - ISI - Ciclo 2026

Fundamentación

El Trabajo Final Integrador tiene como propósito recuperar e integrar los contenidos desarrollados durante el cursado de la asignatura mediante la resolución de una problemática concreta utilizando Python.

La propuesta busca que las y los estudiantes puedan aplicar los conocimientos adquiridos en relación con estructuras de control, modularización, validaciones y resolución algorítmica de problemas, promoviendo además el desarrollo de habilidades vinculadas al pensamiento crítico, la comunicación técnica, el trabajo colaborativo y la toma de decisiones dentro de un equipo de desarrollo.

Asimismo, el trabajo pretende incorporar herramientas y dinámicas propias del desarrollo profesional de software, promoviendo el uso de repositorios Git, la organización progresiva del código y el uso responsable y crítico de herramientas de Inteligencia Artificial como apoyo al aprendizaje y a la programación.

Objetivos

A través del desarrollo del Trabajo Final Integrador se espera que las y los estudiantes logren:

- aplicar estructuras condicionales y repetitivas en la resolución de problemas;
- diseñar soluciones algorítmicas utilizando Python;
- modularizar programas mediante funciones;
- implementar validaciones y manejo básico de errores;
- desarrollar estrategias de resolución y depuración de problemas;
- fortalecer el pensamiento crítico frente a soluciones propuestas por herramientas de Inteligencia Artificial;
- trabajar colaborativamente dentro de un equipo de desarrollo;
- comunicar y defender técnicamente una solución implementada;
- utilizar GitHub o GitLab como herramienta básica de organización y trabajo colaborativo;
- comprender integralmente el funcionamiento del sistema desarrollado.

Modalidad de trabajo

1. El trabajo deberá realizarse en grupos de entre 3 y 5 integrantes, ni más ni menos.
2. Cada equipo desarrollará un sistema basado en uno de los escenarios propuestos por la cátedra. Los escenarios serán asignados al azar por el equipo docente.
3. Todos los sistemas deberán desarrollarse utilizando Python y ejecutarse por consola.
4. La solución implementada deberá contemplar el uso de:
 1. estructuras condicionales;
 2. estructuras repetitivas;
 3. funciones;
 4. validaciones;
 5. acumuladores y contadores;
 6. modularización básica;
 7. manejo básico de errores y mensajes al usuario.
5. Se espera que cada grupo diseñe una solución funcional y coherente con el escenario asignado, incorporando distintas alternativas de interacción y validación de datos.

Escenarios propuestos

1. Sistema de turnos hospitalarios

El sistema deberá permitir administrar la atención de pacientes dentro de un centro de salud. La solución podrá contemplar el registro de pacientes, asignación de turnos, clasificación de prioridad según urgencia, control de pacientes atendidos y generación de estadísticas básicas sobre atención diaria.

El sistema deberá permitir validar datos ingresados, controlar disponibilidad de turnos y gestionar distintos tipos de consultas o especialidades médicas.

2. Sistema de reservas de cine

El sistema deberá administrar reservas de entradas para funciones de cine. Podrá contemplar selección de películas, horarios disponibles, cantidad de entradas, control de capacidad de salas, promociones y cálculo de importes.

La solución podrá incorporar estadísticas básicas relacionadas con películas más elegidas, horarios con mayor demanda o cantidad total de entradas vendidas.

3. Gestión de torneo amateur

El sistema deberá permitir administrar un torneo deportivo amateur. La solución podrá contemplar registro de equipos, carga de resultados, cálculo automático de puntos, posiciones y estadísticas generales del torneo.

El sistema podrá incluir control de partidos jugados, diferencia de goles y generación de tablas de posiciones básicas.

4. Simulador de cajero automático

El sistema deberá simular operaciones básicas de un cajero automático. La solución podrá incluir validación de acceso mediante usuario y contraseña, consulta de saldo, extracción, depósitos y transferencias simples.

También podrá contemplarse control de saldo insuficiente, límites de extracción y registro básico de operaciones realizadas.

5. Sistema de pedidos de comida

El sistema deberá gestionar pedidos dentro de un local gastronómico o servicio de comidas. Podrá incluir selección de productos, armado de pedidos, cálculo de importes, promociones, descuentos y estadísticas de ventas.

El sistema podrá incorporar distintas categorías de productos, medios de pago y control de pedidos realizados durante el día.

6. Gestión de estacionamiento

El sistema deberá administrar el funcionamiento de un estacionamiento. La solución podrá contemplar ingreso y egreso de vehículos, cálculo de tiempo de permanencia, control de espacios disponibles y cálculo de importes a pagar.

También podrá incorporar estadísticas relacionadas con ocupación, cantidad de vehículos atendidos o tiempo promedio de permanencia.

7. Sistema de biblioteca

El sistema deberá gestionar préstamos y devoluciones dentro de una biblioteca. La solución podrá incluir registro de usuarios, préstamos de libros, devoluciones, control de disponibilidad y cálculo de multas simples por demora.

Asimismo, podrá incorporar estadísticas relacionadas con libros más solicitados o cantidad de préstamos realizados.

8. Gestión de gimnasio

El sistema deberá administrar socios y actividades de un gimnasio. Podrá contemplar registro de socios, control de cuotas, inscripción a actividades, control de asistencia y estadísticas básicas de concurrencia.

El sistema también podrá gestionar distintos tipos de membresías o promociones.

9. Sistema de venta de entradas para recitales

El sistema deberá permitir administrar entradas para eventos musicales. La solución podrá contemplar selección de sectores, cálculo de importes, control de capacidad y aplicación de promociones o descuentos.

También podrá incorporar estadísticas de ventas y sectores más demandados.

10. Gestión de hotel

El sistema deberá administrar reservas y ocupación de habitaciones dentro de un hotel. Podrá contemplar registro de huéspedes, check-in, check-out, control de habitaciones disponibles y cálculo de estadías.

La solución también podrá incorporar distintos tipos de habitaciones y estadísticas de ocupación.

11. Sistema de alquiler de bicicletas

El sistema deberá gestionar alquileres de bicicletas dentro de un parque o ciudad. Podrá contemplar registro de clientes, alquileres realizados, cálculo de tiempo de uso y cálculo de importes.

También podrá incorporar control de bicicletas disponibles y estadísticas de utilización.

12. Gestión de supermercado

El sistema deberá simular el funcionamiento básico de una caja de supermercado. La solución podrá incluir carga de productos, cálculo de totales, descuentos, promociones y generación de tickets simples.

También podrá incorporar estadísticas de ventas y productos más vendidos.

13. Sistema de inscripción a cursos

El sistema deberá administrar inscripciones a cursos o talleres. Podrá contemplar registro de estudiantes, control de cupos, asignación de cursos y estadísticas de inscriptos.

Asimismo, podrá incorporar control de listas de espera o cursos con mayor demanda.

14. Gestión de veterinaria

El sistema deberá administrar turnos y atención de mascotas dentro de una veterinaria. La solución podrá incluir registro de mascotas, turnos, atención médica y control de servicios realizados.

También podrá incorporar estadísticas básicas relacionadas con tipos de consultas o servicios más frecuentes.

15. Sistema de delivery

El sistema deberá gestionar pedidos y entregas de un servicio de delivery. La solución podrá contemplar registro de pedidos, cálculo de importes, control de entregas y estadísticas de ventas o zonas de reparto.

Asimismo, podrá incorporar distintos estados de pedido y control de entregas realizadas.

Requisitos técnicos

El trabajo deberá desarrollarse obligatoriamente utilizando Python.

Cada grupo deberá trabajar utilizando un repositorio GitHub o GitLab. El repositorio deberá evidenciar el proceso de construcción del proyecto mediante commits realizados durante el desarrollo y no únicamente una carga final del código terminado.

Además del código fuente, el repositorio deberá incluir un archivo README indicando:

- integrantes del grupo;
- comisión;
- descripción general del sistema;
- instrucciones de ejecución.

Se recomienda mantener el repositorio ordenado y realizar commits periódicos durante el desarrollo del trabajo.

Uso de Inteligencia Artificial

La utilización de herramientas de Inteligencia Artificial se encuentra permitida como herramienta de apoyo al aprendizaje y al desarrollo del proyecto.

Las herramientas de IA podrán utilizarse para:

- generar ideas;
- resolver dudas;
- analizar errores;

- proponer estructuras de código;
- asistir en la depuración;
- o colaborar en el proceso de desarrollo.

Sin embargo, todos los integrantes deberán comprender integralmente la solución implementada y poder justificar las decisiones tomadas durante el desarrollo.

La utilización de IA no reemplaza la necesidad de comprender:

- la lógica implementada;
- las estructuras utilizadas;
- las validaciones realizadas;
- el manejo de errores;
- y el funcionamiento general del sistema.

Se espera además que las y los estudiantes desarrollen una mirada crítica sobre las respuestas generadas por herramientas de IA, pudiendo analizar, adaptar y validar las soluciones propuestas antes de incorporarlas al proyecto.

Deberán registrar en el [README.me](https://github.com/README.me) de Github, qué IA han utilizado, para qué y cómo.

Entregables

La entrega digital se realizará mediante la tarea habilitada por la cátedra solamente el día **miércoles 8/7**.

Cada equipo deberá presentar:

- enlace al repositorio GitHub/GitLab;
- archivo README;
- video de demo del sistema implementado.

El video tendrá una duración de no más de 5' y deberá incluir:

- presentación breve de todos los integrantes;
- explicación general del sistema desarrollado;
- ejecución de al menos un caso de prueba válido;
- ejecución de un caso que contemple validaciones o mensajes de error.

El objetivo de esta instancia es evidenciar el funcionamiento del sistema y el proceso de desarrollo realizado por el grupo.

Modalidad de evaluación

La evaluación del Trabajo Final Integrador se realizará en dos instancias complementarias: entrega digital y coloquio.

Posteriormente a la entrega, **entre el 13 y el 17 de julio**, la cátedra convocará a un coloquio a una selección aleatoria de equipos. La convocatoria será informada oportunamente por los canales oficiales de la asignatura.

Durante el coloquio, uno/a de los integrantes del grupo será seleccionado/a para defender la producción realizada ante un tribunal evaluador conformado por docentes de la cátedra.

En dicha instancia, el/la estudiante deberá demostrar comprensión integral del sistema implementado y responder preguntas vinculadas tanto al funcionamiento del proyecto como a las decisiones tomadas durante el desarrollo.

Asimismo, el equipo docente podrá realizar preguntas vinculadas tanto a la lógica implementada como al proceso de desarrollo del trabajo, incluyendo el uso de herramientas de Inteligencia Artificial.

Algunas preguntas orientativas que podrán realizarse durante la evaluación son:

- ¿Cuál fue el uso de la Inteligencia Artificial durante el desarrollo del proyecto?
- ¿Qué partes del código fueron desarrolladas con asistencia de IA?
- ¿Qué estructuras de control utilizaron y por qué las eligieron?
- ¿Dónde implementaron estructuras repetitivas y qué función cumplen?
- ¿Cómo resolvieron las validaciones de datos ingresados por el usuario?
- ¿Qué tipos de errores contemplaron en el sistema?
- ¿Cómo definieron los mensajes de error mostrados al usuario?
- ¿Qué funciones implementaron y cómo organizaron el sistema?
- ¿Cómo dividieron las responsabilidades dentro del grupo?
- ¿Qué dificultades encontraron durante el desarrollo y cómo las resolvieron?
- ¿Qué mejoras incorporarían al sistema en una próxima versión?
- ¿Qué criterio utilizaron para modularizar el código?
- ¿Qué variables utilizaron como acumuladores o contadores?
- ¿Cómo probaron el correcto funcionamiento del sistema?
- ¿Qué parte del desarrollo consideran más compleja o importante?

La evaluación contemplará tanto el correcto funcionamiento del sistema como la comprensión de la solución implementada, la organización del código y la capacidad de justificar las decisiones tomadas durante el desarrollo.

Consideraciones finales

Se recomienda comenzar el desarrollo del trabajo de manera anticipada, realizar pruebas frecuentes y mantener una organización adecuada del repositorio y del código fuente.

Todos los integrantes deberán involucrarse activamente en el desarrollo del proyecto y conocer integralmente la solución implementada.